

Unidade III

5 COLEÇÕES E GENÉRICOS

Em projetos feitos em C#, muitos desenvolvedores esbarram primeiro em coleções por necessidade prática: é preciso guardar itens de catálogo, mapear identificadores para objetos, percorrer resultados de consultas e aplicar filtros sucessivos, tudo isso sem perder desempenho nem a clareza do código.

Quando essas decisões são tomadas apenas com arrays fixos ou coleções antigas baseadas em `object`, começam a surgir sintomas conhecidos: conversões forçadas de tipo espalhadas pelo código, exceções em tempo de execução por casts incorretos, uso excessivo de boxing e unboxing e dificuldade em evoluir as estruturas de dados sem o risco de erros sutis.

O problema deixa de ser simplesmente armazenar vários elementos e passa a envolver segurança de tipos, custo de acesso e legibilidade das operações de busca, iteração e filtragem.

As coleções genéricas introduzidas no .NET foram pensadas justamente para responder a essas dores, oferecendo estruturas como `List<T>` e `Dictionary<TKey, TValue>`, que combinam type safety em tempo de compilação com implementações eficientes baseadas em arrays redimensionáveis e tabelas hash.



Observação

Uma tabela hash é uma estrutura de dados que armazena pares chave-valor de modo que a chave seja transformada rapidamente em uma posição (índice) de acesso dentro de um array interno. Essa transformação é feita por uma função de hash: dado um objeto usado como chave (por exemplo, uma string com o nome "Alice"), a função calcula um número inteiro, que é então mapeado para uma posição válida do array.

Como o array interno tem tamanho finito e muitas chaves diferentes podem gerar o mesmo índice, surgem colisões. As tabelas hash lidam com isso usando técnicas como encadeamento (cada posição do array guarda uma pequena lista de elementos que caíram naquele índice) ou endereçamento aberto (o algoritmo procura outra posição livre no array conforme uma estratégia predefinida).

No caso do `Dictionary<TKey, TValue>` do .NET, cada chave fornece um código de hash por meio de `GetHashCode()`, e o dicionário usa esse valor para decidir onde armazenar e como buscar o par chave-valor correspondente. Quando se faz `idades["Alice"]`, por exemplo, o dicionário aplica a função de hash sobre a string "Alice", encontra o índice no array interno e, a partir daí, localiza o valor associado.

Ainda assim, o uso dessas coleções genéricas de forma ingênua pode levar a dependências rígidas entre partes do sistema, a métodos que aceitam apenas um tipo específico de lista e a um código que se torna pouco reutilizável à medida que novas necessidades surgem.

A interface `IEnumerable<T>` e o mecanismo de iteração com `foreach` entram nesse cenário como uma camada de abstração: permitem tratar coleções distintas como sequências genéricas, esconder detalhes de armazenamento e até criar sequências virtuais com `yield return`, em que os valores são produzidos sob demanda em vez de serem todos materializados de uma vez.

A partir do momento em que o código passa a ser escrito de forma genérica, surge outra família de problemas, como: restringir os tipos aceitos para garantir que certas operações estarão disponíveis; expressar igualdade e ordenação de maneira consistente; e como fazer isso sem cair em verificações manuais frágeis em tempo de execução. É aí que entram as restrições com `where` e o uso de interfaces como `IEquatable<T>`, `IComparable<T>`, `IComparer<T>` e `IEqualityComparer<T>`, além de padrões de implementação que mantêm `Equals`, `GetHashCode` e `CompareTo` coerentes entre si.

Sem esse cuidado, coleções como `List<T>`, `HashSet<T>` e `Dictionary<TKey, TValue>` podem se comportar de modo inesperado, de forma que duas instâncias que parecem iguais visualmente não são reconhecidas como tal, fazendo com que a ordenação produza resultados confusos e o programador precise recorrer a soluções pontuais que enfraquecem o contrato dos tipos de domínio.

Este tópico se insere justamente nesse conjunto de preocupações, tomando como fio condutor exemplos concretos que podem ser executados no Visual Studio para mostrar como `List<T>`, `Dictionary<TKey, TValue>`, `IEnumerable<T>`, iteradores com `yield` e restrições genéricas se articulam na prática.

5.1 `List<T>`, `Dictionary<TKey, TValue>`, iteração e `IEnumerable<T>`

`List<T>` e `Dictionary<TKey, TValue>` são dois dos tipos de **coleções genéricas** mais comuns no C#. Ambas pertencem ao namespace `System.Collections.Generic`, que reúne coleções capazes de armazenar múltiplos objetos.

No modelo .NET, coleções genéricas oferecem segurança de tipo em tempo de compilação. Por exemplo, uma `List<string>` só aceita objetos `string`, ao contrário das coleções não genéricas antigas, como `ArrayList`, que permitem qualquer objeto sem verificação estática.

O uso de coleções genéricas evita a necessidade de unboxing ou cast desnecessário e previne erros em tempo de execução. Além disso, `List<T>` e `Dictionary<TKey, TValue>` têm implementações eficientes para inserção e busca de elementos, aproveitando internamente os arrays dinâmicos e as tabelas hash, respectivamente. As versões genéricas também implementam `IEnumerable<T>`, possibilitando fácil iteração.

Tipos por valor, como `int`, `double`, `bool` e `DateTime`, são armazenados de forma diferente de tipos por referência, como `string` e classes em geral. Quando um valor desses é colocado em uma variável do tipo `object` ou em uma coleção que guarda um `object`, o runtime precisa embrulhar esse valor em um objeto alocado no heap: isso é o `boxing`. O `unboxing` é o caminho inverso: pegar um `object` que, na verdade, contém um valor empacotado, verificar se o tipo interno é compatível e, se for, copiar de volta para uma variável do tipo por valor correto. Por exemplo, ao adicionar um `int` a um `ArrayList`, ocorre `boxing`; ao recuperar e atribuir de volta a um `int` com algo como `int x = (int) lista[0];`, há `unboxing`. Esse processo tem custo de desempenho (alocação e cópia) e risco de erro em tempo de execução, caso o objeto não contenha o tipo esperado.

Já `cast` é o termo usado para conversão explícita de tipos em C#. Quando você escreve (TipoDesejado) expressão, está dizendo ao compilador para tratar tal valor como sendo de outro tipo, sujeito a verificações em tempo de execução. Em coleções não genéricas que armazenam `object`, toda vez que se recupera um elemento é comum precisar de um `cast` para o tipo concreto – por exemplo, `string nome = (string) arrayList[0];`. Se o objeto ali dentro não for de fato uma `string`, o programa lança uma `InvalidCastException`. Nas coleções genéricas (`List<T>`, `Dictionary<TKey, TValue>`), o parâmetro de tipo `T` já fixa o tipo dos elementos em tempo de compilação, de forma que cada item recuperado da coleção vem com o tipo correto, dispensando `casts` explícitos na maioria dos casos.

Além disso, como `List<int>` armazena diretamente valores `int`, não há `boxing` nem `unboxing` ao adicionar ou ler esses elementos. É por isso que afirmamos há pouco que o uso de coleções genéricas evita `unboxing` ou `cast` desnecessário: ao especializar a coleção para um tipo concreto, o compilador garante a segurança de tipos, elimina conversões em tempo de execução e reduz o risco de erros por conversões inválidas.

Uma limitação dos arrays fixos do C# está em seu tamanho imutável: o problema clássico do tipo `Array` é seu tamanho fixo. Se não soubermos quantos elementos serão armazenados, corremos o risco de declarar uma matriz muito pequena (estourando espaço) ou muito grande (desperdiçando memória).

Em contraste, a classe `List<T>` é projetada como um array cujo tamanho é aumentado dinamicamente conforme necessário, resolvendo esse problema. Na prática, ao criarmos `List<T>` e adicionarmos itens a ela via `List.Add(...)`, seu espaço interno se expande automaticamente. Quando a capacidade atual (propriedade `Capacity`) é ultrapassada pelo número de itens (propriedade `Count`), o `List` aloca um array maior e copia os elementos antigos, garantindo que sempre haja espaço. Por exemplo:

```
List<int> numeros = new List<int>();  
numeros.Add(5);  
numeros.Add(10);  
numeros.Add(15);
```

Nesse código, o objeto `numeros` inicia vazio, e a cada chamada a `Add` a lista cresce automaticamente. As propriedades associadas podem ser acessadas: `Capacity` indica o espaço alocado atual, que é

aumentado automaticamente se `Count` ultrapassar `Capacity`, enquanto `Count` mostra quantos elementos estão presentes.

Assim, se inserirmos elementos suficientes para exceder a capacidade, `List<T>` realoca internamente; caso contrário, podemos reduzir o espaço chamando métodos como `Trim()` para igualar `Capacity` ao valor de `Count`. Dessa forma, o desenvolvedor não precisa fixar antecipadamente o tamanho máximo da coleção.

Para percorrer os itens de uma `List<T>`, utiliza-se normalmente o comando **foreach**. O C# permite que qualquer objeto que implemente `IEnumerable<T>` seja iterado com essa estrutura. `List<T>` já implementa `IEnumerable<T>`, de modo que o exemplo anterior pode ser iterado assim:

```
foreach (int n in numeros) {  
    Console.WriteLine(n);  
}
```

Esse loop imprimirá cada número da lista na ordem em que foi adicionado. O **foreach** funciona nos bastidores chamando `GetEnumerator()` no objeto lista, obtendo um enumerador (`IEnumerator<T>`) que percorre os elementos internamente. Cada iteração avança o enumerador e acessa a propriedade `Current`. Graças a isso, não precisamos conhecer detalhes do armazenamento interno (a lista) ao iterar; basta saber que `List<T>` é enumerável.

Já `Dictionary<TKey, TValue>` é uma coleção genérica de **pares chave-valor**, ou seja, mapeia cada chave a um valor correspondente. Conforme Gunnerson e Wienholt (2012) definem, `Dictionary<TKey, TValue>` é uma coleção de chaves e valores. A chave (`TKey`) serve de índice único; não podem existir duas entradas com a mesma chave.

Na implementação padrão, `Dictionary` usa uma tabela hash interna baseada no método `GetHashCode` das chaves, o que permite acesso rápido pelo valor da chave. Por exemplo, podemos declarar e usar um dicionário assim:

```
Dictionary<string, int> idades = new Dictionary<string, int>();  
idades["Alice"] = 30;  
idades["Bob"] = 25;
```

Aqui, criamos um dicionário que associa nomes (`string`) a idades (`int`). Inserimos ou atualizamos valores definindo `idades["Alice"] = 30`. Para ler, fazemos `int idadeAlice = idades["Alice"]`. Se consultarmos por uma chave inexistente, ocorre exceção ou podemos usar o método `TryGetValue` para evitar erros.

Para iterar um `Dictionary<TKey, TValue>`, podemos também usar **foreach**, mas iteramos sobre pares (do tipo `KeyValuePair<TKey, TValue>`). Por exemplo:

```
foreach (KeyValuePair<string, int> par in idades) {  
    Console.WriteLine($"{par.Key}: {par.Value}");  
}
```

Esse código imprime cada chave e seu valor. Alternativamente, podemos iterar sobre `Keys` (coleção de chaves) ou `Values` (coleção de valores) separadamente:

```
foreach (string chave in idades.Keys) {  
    Console.WriteLine(chave);  
}  
foreach (int valor in idades.Values) {  
    Console.WriteLine(valor);  
}
```

Note-se que `Dictionary<TKey, TValue>` também implementa `IEnumerable<KeyValuePair<TKey, TValue>>`, bem como as interfaces não genéricas correspondentes. Isso significa que todo `Dictionary` é enumerável, permitindo a construção de expressões LINQ ou o uso de estruturas de iteração padrão.

O mecanismo de iteração no C# baseia-se na interface `IEnumerable` (não genérica) e em sua versão genérica `IEnumerable<T>`. Conforme a literatura, para tornar uma classe enumerável, deve-se implementar a interface `IEnumerable` em uma classe.

Na prática, a maioria das coleções genéricas do .NET, incluindo `List<T>` e `Dictionary<TKey, TValue>`, já implementa `IEnumerable<T>`. Isso possibilita usar o **foreach** conforme mencionado, pois esse comando busca `GetEnumerator()` no objeto para obter um `IEnumerator`. Portanto, qualquer coleção que implemente `IEnumerable<T>` pode ser percorrida uniformemente.

Assim, seria possível criar uma classe própria, como uma lista personalizada, implementando `IEnumerable<T>` para torná-la compatível com **foreach**. O exemplo clássico é implementar `IEnumerable<T>` retornando um `IEnumerator<T>` através de um método `GetEnumerator()`. Além disso, o C# fornece o recurso `yield return`, que simplifica a implementação de `IEnumerable<T>` sem escrever a classe enumeradora manualmente. Vejamos o exemplo a seguir:

```
public IEnumerable<int> NumerosImpares(int n) {  
    for(int i = 1; i <= n; i += 2)  
        yield return i;  
}
```

Nesse caso, o método `NumerosImpares` retorna uma sequência enumerável de números ímpares. Podemos percorrê-la com `foreach (int i in NumerosImpares(10)) { ... }`. O compilador transforma esse código em uma classe enumeradora implícita. Esse tipo de implementação baseada em `IEnumerable<T>` é muito útil, pois permite criar sequências virtuais, atrasando (ou *deferring*) a geração de valores até que realmente sejam iterados.

Vamos agora a um exemplo um pouco mais extenso:

```
using System;
using System.Collections.Generic;

namespace ExemploListEDictionary
{
    internal class Program
    {
        static void Main(string[] args)
        {
            // Cria uma lista de produtos (List<Produto>)
            List<Produto> catalogo = new List<Produto>
            {
                new Produto(1, "Teclado mecânico", 350.00m),
                new Produto(2, "Mouse gamer", 150.00m),
                new Produto(3, "Monitor 24\"", 900.00m),
                new Produto(4, "Headset", 250.00m)
            };

            Console.WriteLine("=== Catálogo inicial (List<Produto>) ===");
            ExibirCatalogo(catalogo);

            Console.WriteLine();
            Console.WriteLine($"Count da lista: {catalogo.Count}");
            Console.WriteLine($"Capacity da lista: {catalogo.Capacity}");

            // Adiciona mais um produto: a lista cresce conforme necessário
            catalogo.Add(new Produto(5, "Webcam Full HD", 300.00m));

            Console.WriteLine();
            Console.WriteLine("=== Catálogo após adicionar um produto ===");
            ExibirCatalogo(catalogo);

            Console.WriteLine();
            Console.WriteLine($"Count da lista: {catalogo.Count}");
            Console.WriteLine($"Capacity da lista: {catalogo.Capacity}");

            // Cria um Dictionary<int, Produto> para acesso rápido por Id
            Dictionary<int, Produto> produtosPorId = new Dictionary<int,
Produto>();

            foreach (var produto in catalogo)
            {
                produtosPorId[produto.Id] = produto;
            }

            Console.WriteLine();
            Console.WriteLine("Digite um id de produto para buscar: ");
            string entrada = Console.ReadLine();

            if (int.TryParse(entrada, out int idBuscado))
            {
                // Busca segura com TryGetValue (evita exceção se a chave não
existir)
                if (produtosPorId.TryGetValue(idBuscado, out Produto
produtoEncontrado))
                {

```

```
        Console.WriteLine();
        Console.WriteLine("Produto encontrado no Dictionary:");
        Console.WriteLine(produtoEncontrado);
    }
    else
    {
        Console.WriteLine();
        Console.WriteLine("Nenhum produto encontrado com esse id.");
    }
}
else
{
    Console.WriteLine();
    Console.WriteLine("Id inválido.");
}

Console.WriteLine();
Console.WriteLine("=== Produtos com preço a partir de R$ 300,00
===");

// Usa um método que devolve IEnumerable<Produto> com yield return
foreach (var produto in FiltrarPorPrecoMinimo(catalogo, 300.00m))
{
    Console.WriteLine(produto);
}

Console.WriteLine();
Console.WriteLine("Pressione ENTER para sair.");
Console.ReadLine();
}

// Este método aceita IEnumerable<Produto>, não apenas List<Produto>
// Qualquer coleção genérica que implemente IEnumerable<Produto> pode
ser passada aqui
static void ExibirCatalogo(IEnumerable<Produto> produtos)
{
    foreach (var produto in produtos)
    {
        Console.WriteLine(produto);
    }
}

// Exemplo de iterator que devolve apenas produtos acima de um preço
mínimo
// O retorno é IEnumerable<Produto>, implementado com yield return
static IEnumerable<Produto> FiltrarPorPrecoMinimo(
    IEnumerable<Produto> produtos,
    decimal precoMinimo)
{
    foreach (var produto in produtos)
    {
        if (produto.Preco >= precoMinimo)
        {
            yield return produto;
        }
    }
}
```

```

    }
}

internal class Produto
{
    public int Id { get; }
    public string Nome { get; }
    public decimal Preco { get; }

    public Produto(int id, string nome, decimal preco)
    {
        Id = id;
        Nome = nome;
        Preco = preco;
    }

    public override string ToString()
    {
        return $"{Id} - {Nome} (R$ {Preco:F2})";
    }
}
}

```

Esse código pode ser criado em um projeto de Aplicativo para Console no Visual Studio (ExemploListEDictionary como nome do projeto, por exemplo). O método **Main** é o ponto de entrada do programa e demonstra, em sequência, o uso prático de `List<T>`, de `Dictionary<TKey, TValue>` e de um método que devolve `IEnumerable<T>` usando `yield return`.

A primeira parte do **Main** instancia uma `List<Produto>` chamada `catalogo`. O tipo **Produto** é uma classe definida no final do arquivo, com três propriedades: `Id` (inteiro), `Nome` (string) e `Preco` (decimal). A lista é criada já com quatro elementos, usando inicialização de coleção: cada chamada a `new Produto(...)` cria um objeto com dados de um item de catálogo (teclado, mouse, monitor e headset). Como `List<Produto>` é uma coleção genérica, o compilador garante que só objetos **Produto** podem ser inseridos; não há qualquer conversão para `object` nem necessidade de `cast` na leitura.

Em seguida, o programa escreve no console a frase **“=== catálogo inicial (List<Produto>) ===”** e chama o método auxiliar `ExibirCatalogo(catalogo)`. Esse método recebe um parâmetro do tipo `IEnumerable<Produto>` em vez de `List<Produto>`; isso é intencional e dialoga com o texto teórico: como `List<T>` implementa `IEnumerable<T>`, é possível tratar a lista de maneira genérica, apenas como uma sequência enumerável de produtos.

Dentro de `ExibirCatalogo`, há um `foreach (var produto in produtos)`, que chama o método `GetEnumerator()` nos bastidores e percorre cada elemento, imprimindo-o. A impressão usa o `Console.WriteLine(produto)`, que por sua vez chama o `ToString()` sobrescrito na classe **Produto**. O `ToString()` foi implementado para exibir de forma legível `Id`, `Nome` e `Preco` formatado com duas casas decimais, o que torna a saída no console simples de ler.

Logo depois, o programa mostra os valores de `Count` e `Capacity` da lista. O `Count` indica quantos elementos existem no momento (inicialmente quatro). O `Capacity` representa o tamanho do array interno que a `List<T>` reserva para armazenar seus itens. Graças à implementação interna baseada em um array redimensionável, essa capacidade pode ser maior que o número de elementos; isso permite adicionar novos itens sem realocar a cada inserção.

O código então adiciona mais um produto com `catalogo.Add(new Produto(5, "webcam Full HD", 300.00m))`. Depois da inserção, o catálogo é exibido novamente e os valores de `Count` e `Capacity` são mostrados outra vez, evidenciando que a lista cresceu em número de elementos sem que o desenvolvedor tivesse que se preocupar com o tamanho máximo aprioristicamente.

A etapa seguinte constrói um `Dictionary<int, Produto>` chamado `produtosPorId`. A chave do dicionário é o `int Id` do produto, e o valor é o próprio objeto `Produto`. O `foreach (var produto in catalogo)` percorre cada item da lista e, para cada um, executa `produtosPorId[produto.Id] = produto;`. Essa sintaxe usa o indexador do dicionário: se a chave ainda não existia, a entrada é criada; se já existia, o valor é substituído.

O ponto importante é que `Dictionary<TKey, TValue>` guarda um par chave-valor e, pela implementação baseada em tabela hash, permite localizar rapidamente um produto quando a chave (o `Id`) é conhecida.

Depois de montar o dicionário, o programa interage com o usuário. A linha `Console.WriteLine("Digite um id de produto para buscar: ");` pede que o usuário digite um número. O valor lido por `Console.ReadLine()` é uma string, então o código utiliza `int.TryParse` para tentar convertê-la em inteiro. Se a conversão falha, o programa informa `"Id inválido."` e não tenta acessar o dicionário.

Se a conversão tem sucesso (`out int idBuscado`), o código chama `produtosPorId.TryGetValue(idBuscado, out Produto produtoEncontrado)`. Esse método retorna `true` se houver uma entrada com a chave pedida e, nesse caso, coloca o valor correspondente na variável `produtoEncontrado`.

Dessa forma, a busca é segura: não ocorre exceção caso a chave não exista, em contraste com o uso direto do indexador `produtosPorId[id]`, que dispara uma exceção se a chave não for encontrada. Se `TryGetValue` retornar `true`, o programa mostra o produto encontrado; se retornar `false`, informa que não há produto com aquele `id`.

Na parte final do `Main`, aparece um exemplo de método iterador, conectando o conteúdo prático às interfaces de iteração discutidas neste tópico. A chamada `FiltrarPorPrecoMinimo(catalogo, 300.00m)` devolve um `IEnumerable<Produto>` que contém apenas os produtos com preço maior ou igual a 300. O `foreach` consome esse `IEnumerable<Produto>` normalmente e exibe cada produto retornado.

Do ponto de vista de quem usa, não há diferença em relação a uma lista: o laço **foreach** não sabe se os elementos vêm de uma `List<Produto>`, de um `Dictionary<int, Produto>` ou de um método baseado em **yield return**; ele apenas recebe um enumerador e avança item a item.

O método **FiltrarPorPrecoMinimo** mostra a sintaxe de iterator com **yield return**. Ainda, ele recebe um `IEnumerable<Produto>` genérico e um valor mínimo de preço. O laço interno percorre o parâmetro `produtos`. Para cada produto que atenda à condição `produto.Preco >= precoMinimo`, o código executa **yield return produto**;. O compilador transforma esse método em uma classe interna que implementa `IEnumerable<Produto>` e `IEnumerator<Produto>`, com estado próprio, sem que o programador precise escrever essa classe manualmente.

A sequência é avaliada de forma adiada: os produtos são produzidos sob demanda, à medida que o **foreach** externo solicita o próximo elemento. Isso reforça a ideia de que `IEnumerable<T>` permite criar sequências virtuais que só geram valores quando alguém as itera.

Por fim, a classe **Produto** encapsula os dados do domínio do exemplo. Ela tem três propriedades apenas com **get** (imutáveis do lado de fora) e um construtor que recebe `id`, `nome` e `preco`. Essa modelagem evidencia que as coleções genéricas trabalham com objetos de tipos bem definidos, em vez de referências genéricas para `object`.

A integração com `List<Produto>`, `Dictionary<int, Produto>` e métodos baseados em `IEnumerable<Produto>` deixa claro como as coleções genéricas do .NET oferecem segurança de tipos em tempo de compilação, facilitam a iteração com **foreach** e permitem expressar com clareza operações de busca, filtragem e listagem sem recorrer a unboxing ou casts explícitos.

5.2 Genéricos: restrições, igualdade/ordenação e boas práticas

Genéricos são um recurso central do C# para escrever código que funciona com vários tipos mantendo segurança de tipos e boa performance. Em vez de criar uma classe, método ou interface específica para cada tipo concreto, declaramos um parâmetro de tipo, como `T`, e deixamos o compilador verificar se o uso é consistente.

Para que esse parâmetro de tipo não seja completamente livre, a linguagem oferece restrições genéricas com a palavra-chave `where`, que permitem dizer ao compilador que `T` precisa ser um tipo por referência ou um tipo por valor, implementar uma interface específica, herdar de uma classe base ou possuir um construtor público sem parâmetros. Dessa forma, é possível escrever código genérico que ainda assim faz suposições fortes sobre o tipo usado, com verificação em tempo de compilação em vez de verificações manuais em tempo de execução.

Quando usamos `where T : class`, estamos restringindo `T` a tipos por referência, como classes, interfaces, arrays e `string`. Já `where T : struct` limita `T` a tipos por valor não anuláveis, como `int`, `double` ou `DateTime`. A restrição `where T : new()` exige a presença de um construtor público sem parâmetros, o que possibilita criar instâncias de `T` dentro do código genérico com `new T()`.

Também é possível impor que T derive de uma classe base específica, por exemplo `where T : MinhaClasseBase`, garantindo que todos os T tenham pelo menos os membros definidos nessa base. Por fim, `where T : IMinhaInterface` obriga T a implementar determinada interface, habilitando chamadas diretas aos métodos da interface sem casts.

Essas restrições podem ser combinadas, resultando em algo como `where T : MinhaClasseBase, IComparable<T>, new()`, o que indica ao compilador que T herda de `MinhaClasseBase`, é comparável com instâncias de si mesmo e possui construtor sem parâmetros.

A seguir, tem-se um exemplo completo em C# que pode ser executado em um projeto de Aplicativo para Console no Visual Studio e que demonstra o uso de restrições, bem como interfaces de igualdade e ordenação em genéricos.

```
using System;
using System.Collections.Generic;

namespace GenericosDemo
{
    // Classe base usada como restrição de herança
    public abstract class EntidadeBase
    {
        public int Id { get; set; }
    }

    // Tipo concreto que implementa igualdade e ordenação
    public class Cliente : EntidadeBase, IEquatable<Cliente>,
        IComparable<Cliente>
    {
        public string Nome { get; set; }

        // Igualdade fortemente tipada
        public bool Equals(Cliente other)
        {
            if (ReferenceEquals(null, other)) return false;
            if (ReferenceEquals(this, other)) return true;
            return Id == other.Id && string.Equals(Nome, other.Nome,
                StringComparison.Ordinal);
        }

        // Sobrescreve Equals(object) para ser consistente
        public override bool Equals(object obj)
        {
            return Equals(obj as Cliente);
        }

        // GetHashCode consistente com Equals
        public override int GetHashCode()
        {
            int hashId = Id.GetHashCode();
            int hashNome = Nome == null ? 0 : Nome.GetHashCode();
            return hashId ^ hashNome;
        }
    }
}
```

```
// Ordenação natural por nome
public int CompareTo(Cliente other)
{
    if (other == null) return 1;
    return string.Compare(Nome, other.Nome, StringComparison.Ordinal);
}

public override string ToString()
{
    return $"{Id}: {Nome}";
}
}

// Comparador externo para ordenar Cliente por Id
public class ClientePorIdComparer : IComparer<Cliente>
{
    public int Compare(Cliente x, Cliente y)
    {
        if (ReferenceEquals(x, y)) return 0;
        if (x is null) return -1;
        if (y is null) return 1;
        return x.Id.CompareTo(y.Id);
    }
}

// Comparador de igualdade externo para comparar clientes pelo Nome
public class ClientePorNomeIgualdade : IEqualityComparer<Cliente>
{
    public bool Equals(Cliente x, Cliente y)
    {
        if (ReferenceEquals(x, y)) return true;
        if (x is null || y is null) return false;
        return string.Equals(x.Nome, y.Nome, StringComparison.
OrdinalIgnoreCase);
    }

    public int GetHashCode(Cliente obj)
    {
        return obj.Nome == null
            ? 0
            : obj.Nome.ToUpperInvariant().GetHashCode();
    }
}

// Classe genérica com restrição de herança e new()
public class Repositorio<T> where T : EntidadeBase, new()
{
    private readonly List<T> _itens = new List<T>();

    public void Adicionar(T item)
    {
        _itens.Add(item);
    }

    // Demonstra o uso de new() para criar instâncias de T
    public T CriarENovo()
```

```
{
    var entidade = new T(); // Permitido por causa da restrição new()
    _itens.Add(entidade);
    return entidade;
}

public IEnumerable<T> Todos()
{
    return _itens;
}
}

public static class UtilitariosGenericos
{
    // Método genérico que precisa de ordenação: T implementa IComparable<T>
    public static T EncontrarMaximo<T>(IEnumerable<T> itens) where T :
IComparable<T>
    {
        bool primeiro = true;
        T maximo = default(T);

        foreach (var item in itens)
        {
            if (primeiro)
            {
                maximo = item;
                primeiro = false;
            }
            else if (item.CompareTo(maximo) > 0)
            {
                maximo = item;
            }
        }

        if (primeiro)
            throw new InvalidOperationException("Sequência vazia.");

        return maximo;
    }
}

internal class Program
{
    private static void Main(string[] args)
    {
        // Uso de Repositorio<T> com restrições: T deve ser EntidadeBase e
ter new()
        var repositorio = new Repositorio<Cliente>();

        repositorio.Adicionar(new Cliente { Id = 2, Nome = "Maria" });
        repositorio.Adicionar(new Cliente { Id = 1, Nome = "João" });
        repositorio.Adicionar(new Cliente { Id = 3, Nome = "Ana" });

        // Criação de instância via new() dentro do genérico
        var clienteGerado = repositorio.CriarENovo();
        clienteGerado.Id = 4;
    }
}
```

```

clienteGerado.Nome = "Gerado com new()";

Console.WriteLine("Clientes cadastrados no repositório (ordem de
inserção):");
foreach (var c in repositorio.Todos())
{
    Console.WriteLine(c);
}

// Convertendo para List<Cliente> para usar Sort e Contains
var clientes = new List<Cliente>(repositorio.Todos());

// Ordenação natural por Nome usando IComparable<Cliente>
clientes.Sort();
Console.WriteLine();
Console.WriteLine("Clientes ordenados por nome (IComparable<T>:");
foreach (var c in clientes)
{
    Console.WriteLine(c);
}

// Ordenação customizada por Id usando IComparer<Cliente> externo
clientes.Sort(new ClientePorIdComparer());
Console.WriteLine();
Console.WriteLine("Clientes ordenados por Id (IComparer<T>
externo):");
foreach (var c in clientes)
{
    Console.WriteLine(c);
}

// Uso de método genérico que exige IComparable<T>
var maxPorNome = UtilitariosGenericos.EncontrarMaximo(clientes);
Console.WriteLine();
Console.WriteLine($"Maior cliente segundo CompareTo (nome):
{maxPorNome}");

// Uso de IEqualityComparer<T> externo em um HashSet<T>
var conjuntoPorNome = new HashSet<Cliente>(new
ClientePorNomeIgualdade());
conjuntoPorNome.Add(new Cliente { Id = 10, Nome = "JOÃO" });
conjuntoPorNome.Add(new Cliente { Id = 11, Nome = "joão" });

Console.WriteLine();
Console.WriteLine("Quantidade de clientes distintos em HashSet por
nome (IEqualityComparer<T>:");
Console.WriteLine(conjuntoPorNome.Count); // Deve ser 1, pois nomes
equivalem ignorando maiúsculas/minúsculas

// Uso de IEquatable<Cliente> em List<T>.Contains
var procurado = new Cliente { Id = 2, Nome = "Maria" };
bool contem = clientes.Contains(procurado);
Console.WriteLine();
Console.WriteLine($"Lista contém cliente igual a (Id=2, Nome=Maria)?
{contem}");

```

```
        Console.WriteLine();  
        Console.WriteLine("Pressione ENTER para sair.");  
        Console.ReadLine();  
    }  
}
```

Esse programa mostra, em um único executável de console, como as restrições genéricas influenciam o que é possível fazer dentro de uma classe ou método genérico e como as interfaces de igualdade e ordenação se encaixam nesse cenário.

A classe `Repositorio<T>` ilustra o uso de `where T : EntidadeBase, new()`. A restrição de herança garante que todo objeto `T` possui ao menos a propriedade `Id`, herdada de `EntidadeBase`, e a restrição `new()` autoriza a criação de objetos `T` com `new T()`. Isso permite ter métodos como `CriarENovo`, que fabrica instâncias do tipo genérico sem conhecer o tipo concreto em tempo de escrita do código.

No `Main`, o repositório é instanciado como `Repositorio<Cliente>`, o que respeita as restrições: `Cliente` deriva de `EntidadeBase` e possui construtor público sem parâmetros.

A classe `Cliente` demonstra o uso de `IEquatable<Cliente>` e `IComparable<Cliente>` em um tipo de domínio. `IEquatable<Cliente>` expõe um método `Equals(Cliente other)` que compara duas instâncias de `Cliente` diretamente. A implementação apresentada considera dois clientes iguais quando `Id` e `Nome` coincidem.

O código também sobrescreve `Equals(object)` e `GetHashCode` de forma consistente, de modo que coleções genéricas que dependem de igualdade, como `List<Cliente>`, `Contains` e `HashSet<Cliente>`, funcionem corretamente. Isso é usado no final do `Main`, ao criar um objeto procurado com os mesmos dados de uma instância já presente na lista e verificar se `clientes.Contains(procurado)` retorna `true`; o método `Contains` chama a versão tipada de `Equals`, evitando casts e, em tipos por valor, evitando boxing desnecessário.

A interface `IComparable<Cliente>` fornece o método `CompareTo(Cliente other)`, usado para definir a ordenação natural dos objetos `Cliente`. No exemplo, a comparação é feita com base no `Nome`. Quando o código chama `clientes.Sort()`, a classe `List<T>` utiliza `IComparable<T>` para descobrir como ordenar os elementos.

Também há um método genérico em `UtilitariosGenericos`, chamado `EncontrarMaximo<T>`, que varre uma sequência de `T` e devolve o maior elemento conforme `CompareTo`. A cláusula `where T : IComparable<T>` é essencial: sem ela, o compilador não permitiria a chamada `item.CompareTo(maximo)`. Ao passar a lista de clientes para `EncontrarMaximo`, obtemos o cliente cujo nome é considerado o maior segundo a ordenação definida.

A classe `UtilitariosGenericos` foi declarada como estática, isto é, como um tipo que não pode ser instanciado com `new`, não pode ser herdado e contém apenas membros estáticos. Em C#, uma

classe estática (falamos brevemente sobre ela anteriormente) atua como um contêiner de funções auxiliares sem estado, deixando explícito que não existe identidade de objeto nem dados de instância a serem mantidos.

Isso significa, por exemplo, que você nunca escreve `new Console()` ou `new Math()`; em vez disso, chama os métodos diretamente como `Console.WriteLine(...)` ou `Math.Abs(...)` considerando que essas classes são estáticas. Ou seja, não faz sentido criar um objeto de `UtilitariosGenericos`; o método `EncontrarMaximo<T>` é acessado diretamente pelo nome da classe, como em `UtilitariosGenericos.EncontrarMaximo(clientes)`, o que dispensa instâncias supérfluas e reforça, em nível de compilação, que todos os membros são puramente estáticos.

O compilador impede construções como `new UtilitariosGenericos()` e proíbe a declaração de membros de instância, evitando usos equivocados e tornando o papel da classe mais claro: servir apenas como agrupador de utilitários genéricos que processam coleções.

Em muitos cenários, é necessário um critério de ordenação diferente do padrão. Para isso, existe `IComparer<T>`, que define um método `Compare(Tx, Ty)`. A classe `ClientePorIdComparer` implementa `IComparer<Cliente>` ordenando clientes pelo campo `Id`. Quando `clientes.Sort(new ClientePorIdComparer())` é executado, a lista é reordenada com base nesse comparador externo, sem alterar a ordenação natural por nome definida em `Cliente.CompareTo`. Essa separação permite manter uma única definição de ordenação padrão na classe, ainda assim atendendo a necessidades pontuais de ordenações alternativas por meio de comparadores distintos.

Já a interface `IEqualityComparer<T>` permite definir uma lógica de igualdade de fora do tipo. Em `ClientePorNomeIgualdade`, a comparação considera apenas o campo `Nome`, ignorando diferenças de maiúsculas e minúsculas, e `GetHashCode` é calculado de forma consistente com esse critério. Esse comparador é usado ao criar um `HashSet<Cliente>`, de modo que duas instâncias com nomes equivalentes, como "JOÃO" e "joão", sejam tratadas como o mesmo elemento. O resultado de `conjuntoPorNome.Count` é um, ainda que duas instâncias diferentes tenham sido adicionadas, o que ilustra o papel de `IEqualityComparer<T>` na definição de igualdade em coleções baseadas em hashing, como `HashSet<T>` e `Dictionary<TKey, TValue>`.

A estrutura dos tipos genéricos no exemplo também introduz boas práticas. O parâmetro de tipo de `Repositorio<T>` é um único tipo genérico, por isso a letra simples `T` é suficiente, enquanto tipos auxiliares que representam comparadores recebem nomes descritivos (`ClientePorIdComparer`, `ClientePorNomeIgualdade`), condizentes com a recomendação de nomear parâmetros de tipo e classes de forma clara.

Restrições com `where` são utilizadas no lugar de verificações com `is` ou `as` dentro dos métodos, o que deixa o contrato do tipo explícito e faz o compilador impedir usos indevidos. O emprego de interfaces como `IEquatable<T>` e `IComparable<T>` também evita boxing em tipos por valor e torna o código que manipula coleções genéricas mais eficiente e claro.

Finalmente, o exemplo mostra que genéricos são particularmente úteis quando a mesma lógica se aplica a vários tipos, mas que o desenho das interfaces deve ser cuidadoso: o repositório genérico trabalha apenas com `EntidadeBase` e com tipos que possam ser instanciados via `new()`, evitando uma generalização exagerada e mantendo a interface simples de entender e usar.

6 LINQ ESSENCIAL

O LINQ (Language-Integrated Query) é uma das tecnologias mais novas e poderosas de desenvolvimento da Microsoft, que integra consultas diretamente nas linguagens .NET, como o C#. O nome é dado ao conjunto de recursos que integra **operações de consulta diretamente à linguagem**, permitindo escrever consultas sobre coleções, bancos de dados, XML e outras fontes usando sintaxe tipada em C# em vez de strings soltas.

Do ponto de vista técnico, o LINQ se apoia em duas peças centrais: as chamadas query expressions (a sintaxe `from... where... select...` embutida na linguagem) e os standard query operators, que são métodos de extensão definidos principalmente nas classes `System.Linq.Enumerable` e `System.Linq.Queryable`.

Esses operadores atuam sobre sequências que implementam `IEnumerable<T>` ou `IQueryable<T>` e oferecem operações como **filtragem, projeção, ordenação, agrupamento e agregação**. Em outras palavras, quando se escreve uma expressão de consulta em C#, o compilador traduz essa sintaxe em chamadas encadeadas a métodos como `Where`, `Select`, `OrderBy`, `GroupBy` e assim por diante, que formam o núcleo do LINQ. Para visualizar isso em código, considere um exemplo simples com LINQ to Objects, que opera sobre coleções em memória:

```
using System;
using System.Collections.Generic;
using System.Linq;

namespace DemoLinqIntro
{
    internal static class Program
    {
        private static void Main()
        {
            List<int> numeros = new() { 1, 2, 3, 4, 5, 6, 7, 8 };

            // Sintaxe de consulta
            IEnumerable<int> consulta =
                from n in numeros
                where n % 2 == 0
                orderby n descending
                select n * n;

            Console.WriteLine("Resultados da consulta LINQ:");
            foreach (int valor in consulta)
            {
                Console.WriteLine(valor);
            }
        }
    }
}
```

```

        Console.WriteLine("Pressione ENTER para sair.");
        Console.ReadLine();
    }
}

```

Nessa forma, parece que foi criada uma minilinguagem de consulta dentro da linguagem, mas, segundo a especificação, o compilador converte esse trecho em chamadas aos métodos de extensão correspondentes na classe `Enumerable`.

É nesse ponto que as **expressões lambda** entram em cena. Os standard query operators são métodos que recebem funções como parâmetros: por exemplo, `Where` recebe um `Func<TSource, bool>` (um predicado), `Select` recebe um `Func<TSource, TResult>` (uma função de projeção) e `Aggregate` recebe um acumulador `Func<TAccumulate, TSource, TAccumulate>`.

Em C#, a forma concisa e idiomática de fornecer essas funções é por meio de expressões lambda, definidas na referência da linguagem como o mecanismo sintático para criar funções anônimas com o operador `=>`. Se reescrevemos o exemplo anterior usando a chamada sintaxe de métodos, o papel das lambdas fica claro:

```

using System;
using System.Collections.Generic;
using System.Linq;

namespace DemoLinqLambda
{
    internal static class Program
    {
        private static void Main()
        {
            List<int> numeros = new() { 1, 2, 3, 4, 5, 6, 7, 8 };

            IEnumerable<int> consulta = numeros
                .Where(n => n % 2 == 0)           // Lambda como predicado
                .OrderByDescending(n => n)        // Lambda como chave de
ordenação
                .Select(n => n * n);              // Lambda como projeção

            Console.WriteLine("Resultados da consulta LINQ com lambdas:");
            foreach (int valor in consulta)
            {
                Console.WriteLine(valor);
            }

            Console.WriteLine("Pressione ENTER para sair.");
            Console.ReadLine();
        }
    }
}

```

Aqui, cada ocorrência de `n => ...` é uma expressão lambda. O compilador, seguindo as regras descritas na documentação, infere os tipos de parâmetro e retorno a partir da assinatura do método de extensão chamado (`Where`, `OrderByDescending`, `Select`) e converte cada lambda em um delegate `Func<...>` compatível. Dessa forma, as expressões lambda são o meio pelo qual se passa lógica de consulta para os operadores LINQ sem precisar declarar métodos nomeados separados.

Você deve ter observado que, para o LINQ, o `.` (dot operator) também serve para chamar métodos de extensão. A página de operadores de acesso de membro registra que você pode usar o ponto para acessar um método de extensão, o que explica chamadas como `numeros.Where(...).Select(...);`, em que `Where` e `Select` são métodos de extensão definidos na classe estática `System.Linq.Enumerable`.

Esses métodos aparecem para você como se fossem membros do tipo `List<int>` ou `IEnumerable<int>`, justamente porque o compilador trata a expressão `numeros.Where(...)` como uma forma abreviada de `Enumerable.Where(numeros, ...)`, e o operador `.` é o mecanismo sintático para essa chamada encadeada.

Quando se utiliza LINQ sobre coleções em memória (`IEnumerable<T>`), como nos exemplos anteriores, essas lambdas são delegates comuns e o pipeline de operadores é executado no próprio CLR, geralmente com execução adiada: muitos operadores, como `Where` e `Select`, só executam quando a sequência é enumerada (por exemplo, em um `foreach`), característica descrita pela própria Microsoft ao classificar os operadores em execução imediata e adiada.



Saiba mais

O manual a seguir tem como foco apresentar o LINQ, isto é, a ideia de consultas tipadas integradas à linguagem, aplicáveis a coleções em memória, banco de dados (LINQ to SQL) e XML.

ALBAHARI, J.; ALBAHARI, B. *LINQ pocket reference*. 1. ed. Sebastopol: O'Reilly Media, 2008.

Dito de outro modo, uma expressão lambda é a sintaxe usada para declarar uma função anônima, ou seja, uma função sem nome, construída com o operador `=>`, que separa a lista de parâmetros do corpo da função. A documentação oficial descreve exatamente essa forma: parâmetros à esquerda, expressão ou bloco de código à direita; o compilador converte essa construção em um **delegate** apropriado.



Observação

Em C#, delegate é um tipo que representa uma referência para método com uma assinatura específica: tipos de parâmetros e tipo de retorno. A documentação da linguagem descreve delegate como um tipo que encapsula um método, de forma semelhante a um ponteiro para função seguro e compatível com o sistema de tipos do .NET.

Quando dizemos que o compilador converte essa construção em um delegate apropriado, a ideia é a seguinte: toda expressão lambda precisa ter um tipo-alvo (por exemplo, `Func<int, int>` ou `Action<string>`). Esses tipos (`Func<>`, `Action<>`) são, na verdade, tipos delegate genéricos definidos na biblioteca de classes. O compilador olha para a assinatura esperada (parâmetros e retorno) e transforma a lambda em uma instância desse delegate.

No nível da linguagem, há duas formas básicas. A primeira é a **expression lambda**, cujo corpo é uma única expressão e cujo valor é o resultado retornado, por exemplo, `(x, y) => x + y`. A segunda é a **statement lambda**, na qual o corpo é um bloco entre chaves, com múltiplas instruções e, se necessário, uma instrução `return`, como em `(x, y) => { var soma = x + y; return soma; }`. Em ambos os casos, a regra é a mesma: declarar a lista de parâmetros entre parênteses, seguida de `=>` e do corpo.

Toda expressão lambda em C# precisa ter um tipo de destino, que é um tipo delegate. A regra descrita na especificação e na documentação é que, se a lambda não retorna valor, ela é compatível com um `Action<...>`; se retorna valor, é compatível com um `Func<..., TResult>`. Assim, uma lambda com dois parâmetros inteiros e retorno inteiro pode ser atribuída a um `Func<int, int, int>`, ao passo que uma lambda que apenas escreve algo no console, sem retorno, pode ser armazenada em um `Action`.

Um primeiro exemplo mostra o uso mais direto: atribuir uma expressão lambda a um delegate `Func<T, TResult>` e executá-la em um programa de console.

```
// Arquivo Program.cs
using System;

namespace LambdaDemoBasico
{
    internal static class Program
    {
        private static void Main(string[] args)
        {
            // Lambda que calcula o quadrado de um inteiro
            Func<int, int> quadrado = x => x * x;
        }
    }
}
```

```
Console.Write("Informe um número inteiro: ");
string? entrada = Console.ReadLine();

if (!int.TryParse(entrada, out int valor))
{
    Console.WriteLine("Valor inválido.");
    return;
}

int resultado = quadrado(valor);
Console.WriteLine($"O quadrado de {valor} é {resultado}.");

Console.WriteLine("Pressione ENTER para sair.");
Console.ReadLine();
}
}
```

Nessa aplicação, o tipo do parâmetro `x` e o tipo de retorno são inferidos a partir de **Func<int, int>**, conforme a regra padrão de inferência de tipos de lambdas descrita na referência da linguagem.

Expressões lambda aparecem o tempo todo em LINQ, porque os standard query operators (métodos como `Where`, `Select`, `OrderBy`, `Aggregate`) são declarados como métodos de extensão que recebem funções como parâmetros, geralmente escritas como lambdas. Considere o seguinte exemplo, que você pode executar como Aplicativo para Console no Visual Studio 2026:

```
// Arquivo Program.cs
using System;
using System.Collections.Generic;
using System.Linq;

namespace LambdaDemoLinq
{
    internal static class Program
    {
        private static void Main(string[] args)
        {
            List<int> numeros = new()
            {
                5, 12, 7, 3, 18, 21, 2, 10
            };

            // Lambda usada como predicado em Where
            IEnumerable<int> paresMaioresQueDez =
                numeros.Where(n => n % 2 == 0 && n > 10);

            // Lambda usada como projeção em Select
            IEnumerable<string> descricoes =
                paresMaioresQueDez.Select(n => $"Par filtrado: {n}");

            Console.WriteLine("Números originais:");
            Console.WriteLine(string.Join(", ", numeros));
        }
    }
}
```

```

        Console.WriteLine();
        Console.WriteLine("Pares > 10 (filtrados com Where):");
        Console.WriteLine(string.Join(", ", paresMaioresQueDez));

        Console.WriteLine();
        Console.WriteLine("Descrições (projetadas com Select):");
        foreach (string texto in descricoes)
        {
            Console.WriteLine(texto);
        }

        Console.WriteLine();
        Console.WriteLine("Pressione ENTER para sair.");
        Console.ReadLine();
    }
}

```

Nesse código, `n => n % 2 == 0 && n > 10` é um predicado que o método `Where` recebe como argumento, e `n => $"Par filtrado: {n}"` é uma função de projeção que `Select` usa para transformar cada inteiro em uma string de saída. Veremos detalhes sobre esses operadores em breve.

Outro aspecto central das expressões lambda em C# é a captura de variáveis externas, conhecida como **closure**. A documentação explica que uma lambda pode referir-se a variáveis que estavam no escopo do método que a definiu; essas variáveis são chamadas de variáveis externas e, quando capturadas, têm seu tempo de vida estendido para que o delegate resultante possa acessá-las com segurança. Isso permite escrever exemplos como o que vemos a seguir, em que uma variável local define um limite que a lambda utiliza:

```

using System;
using System.Collections.Generic;
using System.Linq;

namespace LambdaDemoClosure
{
    internal static class Program
    {
        private static void Main(string[] args)
        {
            Console.Write("Informe um limite mínimo (inteiro): ");
            string? entrada = Console.ReadLine();

            if (!int.TryParse(entrada, out int limite))
            {
                Console.WriteLine("Valor inválido.");
                return;
            }

            List<int> valores = new() { 3, 7, 10, 15, 22, 5, 30 };

            // A lambda captura a variável 'limite' do escopo externo
            Func<int, bool> maiorOuIgualAoLimite = n => n >= limite;

```

```
var filtrados = valores.Where(maiorOuIgualAoLimite).ToList();

Console.WriteLine();
Console.WriteLine($"Valores originais: {string.Join(", ",
valores)}");
Console.WriteLine($"Valores >= {limite}: {string.Join(", ",
filtrados)}");

Console.WriteLine();
Console.WriteLine("Pressione ENTER para sair.");
Console.ReadLine();
    }
}
```

A variável `limite` é declarada no corpo de `Main`, mas o lambda `n => n >= limite` continua a utilizá-la ao longo da execução da consulta LINQ. O compilador transforma esse padrão em uma estrutura auxiliar interna que garante que `limite` seja armazenado em um campo de objeto acessível tanto pelo método quanto pela expressão lambda, prolongando a vida útil da variável para além do ponto no qual ela sairia de escopo em um código sem captura.

Esses exemplos cobrem o uso típico de expressões lambda em C# moderno: declaração de funções anônimas com `=>`, conversão automática para delegates `Func` e `Action`, participação em consultas LINQ e captura de variáveis externas via closures.



Saiba mais

O livro a seguir, muito detalhado, apresenta um tratamento sistemático das lambdas do ponto de vista tanto sintático quanto semântico, tornando-o uma boa base teórica e prática.

SKEET, J. *C# in depth*. 4. ed. Shelter Island: Manning Publications, 2019.

A obra seguinte funciona como complemento: trabalha expressões lambda em C# dentro de um estilo funcional mais amplo (funções de primeira classe, composição, pipelines etc.), o que ajuda a sair do uso apenas sintático das lambdas e passar a utilizá-las como unidade central de organização do código.

PAINTER, S. J. *Functional programming with C#*: create more supportable, robust, and testable code. Sebastopol: O'Reilly Media, 2023.

Já em cenários como LINQ to Entities (Entity Framework) ou outros provedores que expõem `IQueryable<T>`, a história é ligeiramente diferente. Nesses casos, certas expressões lambda são convertidas não em delegates, mas em árvores de expressão (`Expression<TDelegate>`), estruturas que representam o código da consulta como uma árvore em que cada nó é uma operação (comparação, chamada de método, acesso a propriedade etc.).



Observação

Entity Framework é um framework de mapeamento objeto-relacional (ORM, do inglês object-relational mapping) da Microsoft para a plataforma .NET. A documentação oficial o define como um mapeador de relação de objeto que permite criar uma camada de acesso a dados de alto nível em aplicações .NET usando objetos de domínio em vez de lidar diretamente com comandos SQL e APIs de baixo nível. Em termos práticos, você modela suas entidades como classes .NET, e o Entity Framework cuida de transformar operações sobre esses objetos (consultas, inserções, atualizações, deleções) em comandos apropriados para o banco de dados relacional correspondente.

O LINQ foi concebido para ter vários provedores diferentes, todos usando a mesma sintaxe de consulta, mas conectados a fontes de dados distintas. As principais variações são: LINQ to Objects, LINQ to SQL, LINQ to Entities, LINQ to XML e LINQ to DataSet. **LINQ to Objects** é o caso básico em que se consultam coleções que implementam `IEnumerable<T>`, como `List<T>` ou arrays; a consulta é executada localmente, no CLR, usando delegates comuns.



Lembrete

Delegate é um tipo que representa uma referência para método com uma assinatura específica ou, nas palavras da documentação da linguagem, um tipo que encapsula um método.

LINQ to SQL e **LINQ to Entities**, por sua vez, usam LINQ para acessar dados relacionais. No primeiro caso, o provedor converte a expressão LINQ em SQL para SQL Server, fazendo o mapeamento entre classes e tabelas; no segundo, LINQ to Entities funciona como linguagem de consulta do Entity Framework, na qual as entidades são objetos mapeados ao banco por meio do mapeamento objeto-relacional (ORM) e as consultas são traduzidas internamente para comandos compreendidos pelo provedor de banco de dados.

LINQ to XML atua sobre documentos XML representados como objetos `XDocument` e `XElement`, e **LINQ to DataSet** aplica a mesma sintaxe de consulta a dados carregados em `DataSet` e `DataTable`, trazendo para o mundo ADO.NET tradicional a mesma forma de escrever consultas declarativas.

Além dessas variações principais, a documentação do .NET descreve o PLINQ (**Parallel LINQ**), que não é uma fonte de dados diferente, e sim um mecanismo para executar consultas LINQ de forma paralela sobre sequências `IEnumerable<T>`, por meio do tipo `ParallelQuery<T>` e do método de extensão `AsParallel`.

Nesse cenário, a mesma consulta escrita com operadores LINQ pode ser distribuída em várias threads quando faz sentido, mantendo a sintaxe e o modelo conceitual, mas trocando o plano de execução.

Existem ainda outros provedores de terceiros (por exemplo, para SharePoint ou serviços específicos), construídos sobre a mesma ideia que a arquitetura de LINQ expõe: o compilador gera árvores de expressão ou delegates, e um provedor analisa essas estruturas para executar a consulta na fonte de dados correspondente.

A sintaxe de consulta é uniforme entre os diversos provedores, de forma que o compilador traduz expressões com palavras-chave como `from`, `where`, `select`, `join` e `orderby` em chamadas a operadores padrão de consulta, e essa tradução independe de o alvo ser LINQ to Objects, LINQ to SQL, LINQ to Entities, LINQ to XML ou LINQ to DataSet.

Para o programador C#, isso se traduz no fato de que o conjunto de palavras-chave usadas em expressões de consulta é o mesmo em todas essas variantes e de que a sintaxe alternativa em forma de métodos, com lambdas em chamadas como `colecao.Where(x => ...)`, usa os mesmos operadores definidos nas classes `Enumerable` e `Queryable`, que atuam sobre sequências que implementam `IEnumerable<T>` ou `IQueryable<T>`.

As diferenças concretas entre as variantes de LINQ aparecem sobretudo nas bibliotecas envolvidas e na natureza das fontes de dados. A documentação da Microsoft descreve LINQ to Objects como consultas sobre coleções que implementam `IEnumerable<T>`, usando os operadores de `System.Linq`, enquanto LINQ to Entities e LINQ to SQL trabalham sobre fontes que implementam `IQueryable<T>`, expostas em tipos como `DbSet<T>` no namespace `System.Data.Entity` e em mapeamentos de tabelas via `System.Data.Linq` para LINQ to SQL.

No caso de LINQ to XML, a interface se baseia em tipos como `XDocument` e `XElement`, no namespace `System.Xml.Linq`, com métodos específicos como `Elements` e `Descendants` para percorrer a árvore XML.

Já LINQ to DataSet atua sobre `DataSet` e `DataTable`, que são convertidos em sequências de `DataRow` por meio de `AsEnumerable`, e faz uso de extensões como `Field<T>` para acesso tipado às colunas.

Nos provedores que geram SQL, como LINQ to Entities e LINQ to SQL, as expressões de consulta precisam ser traduzidas para uma árvore de comandos ou para SQL; por isso, apenas um subconjunto das operações e dos métodos do .NET é aceito dentro da expressão, e tentativas de usar métodos que não podem ser mapeados para o lado do servidor resultam em exceções `NotSupportedException` com mensagens do tipo "this method cannot be translated into a store expression", algo amplamente documentado em guias de Entity Framework e discussões técnicas.

6.1 Where/Select/GroupBy/Any, projeções e diferimento de execução

O LINQ permite escrever **consultas sobre coleções de dados** de forma concisa e fortemente tipada, como se fossem parte da própria linguagem. Em LINQ, operadores de consulta padrão (query operators) criam novas sequências a partir das origens de dados originais, sem alterar os objetos fonte.

Esses operadores podem ser encadeados fluentemente, de modo que os dados fluem da esquerda para a direita através da cadeia. Por exemplo, os operadores `Where`, `OrderBy` e `Select` podem ser combinados para filtrar, ordenar e projetar elementos, sempre retornando novas coleções enquanto preservam a fonte original. Essa abordagem funcional inspira-se diretamente no paradigma de programação funcional, garantindo que a coleção original não seja modificada pelas consultas.

```
int[] numeros = { 1, 2, 3, 4, 5, 6 };
var pares = numeros.Where(n => n % 2 == 0);
foreach (var n in pares)
    Console.WriteLine(n);
// Saída esperada: 2 4 6
```

Nesse trecho, o operador `Where` filtra a sequência de inteiros, selecionando somente os números pares. Como se observa, `Where` recebe uma expressão lambda `n => n % 2 == 0` que define a condição booleana de filtragem. Segundo Albahari e Albahari (2008), o operador `Where` emite uma versão filtrada de sua sequência de entrada. Ou seja, apenas elementos que satisfazem a condição (nesse caso, números pares) permanecem no resultado.



Lembrete

Vimos no início deste tópico os formatos de expressão lambda, que são expression lambda e statement lambda.

Assim, o resultado de `Where` é uma nova coleção (do tipo `IEnumerable<int>`) contendo apenas os elementos filtrados. É importante notar que a coleção original `numeros` não sofre alteração; o método `Where` retorna uma nova sequência filtrada.

O operador `Select` do LINQ serve para projeção, ou seja, transformação dos elementos de uma sequência em outra forma ou tipo. Cada elemento de entrada é mapeado para um elemento de saída por meio de uma expressão lambda passada a `Select`. Por exemplo, podemos converter uma sequência de nomes para letras maiúsculas:

```
string[] nomes = { "ana", "beatriz", "carlos" };
var maiusculos = nomes.Select(n => n.ToUpper());
foreach (var n in maiusculos)
    Console.WriteLine(n);
// Saída esperada: ANA, BEATRIZ, CARLOS
```

Aqui, `Select` recebe a expressão `n => n.ToUpper()`, que transforma cada string para maiúsculas. O operador `Select` emite uma sequência na qual cada elemento de entrada é transformado ou projetado segundo a expressão lambda fornecida.

Isso significa que `Select` gera uma nova sequência cujos elementos são o resultado da aplicação dessa transformação. Em outras palavras, ele projeta cada elemento original em um novo elemento. Essa projeção pode criar tipos anônimos ou instâncias de outros tipos; por exemplo, podemos projetar somente certos campos de um objeto em uma coleção de objetos simples. Em todos os casos, a coleta resultante de `Select` é uma nova coleção transformada, deixando inalterada a coleção de origem.

Os operadores `Where`, `Select` e outros do LINQ podem ser encadeados. Por exemplo, imagine filtrar um conjunto de strings para aquelas que contêm a letra "a" e, em seguida, converter o resultado para maiúsculas. Isso pode ser feito em um único pipeline:

```
string[] nomes = { "Tom", "Dick", "Harry", "Mary", "Jay" };
var query = nomes
    .Where(n => n.Contains("a"))
    .Select(n => n.ToUpper());
foreach (var nome in query)
    Console.WriteLine(nome + " ");
// Exemplo de saída: "HARRY MARY JAY"
```

Nesse exemplo, primeiro `.Where(n => n.Contains("a"))` filtra os nomes contendo "a", depois `.Select(n => n.ToUpper())` projeta cada um desses resultados em letras maiúsculas, sendo que os operadores `Where` e `Select` são métodos de extensão na classe `Enumerable`. O fluxo de dados ocorre em etapas: primeiro se filtra (`Where`) e depois se projeta (`Select`). Em qualquer ponto, a coleção original `nomes` não é modificada, pois cada operador retorna uma nova sequência, aplicando sua operação ao resultado do anterior.

Além dos operadores de filtragem e projeção, o LINQ oferece o poderoso operador de agrupamento `GroupBy`, que organiza sequências em grupos baseados em uma chave comum. Ele organiza uma sequência de entrada plana em sequências de grupos. Por exemplo, suponha um array de números e considere que desejemos agrupar os números pares e ímpares separadamente:

```
int[] numeros = { 1, 2, 3, 4, 5, 6 };
var grupos = numeros.GroupBy(n => (n % 2 == 0) ? "Par" : "Ímpar");
foreach (var grupo in grupos) {
    Console.WriteLine("Grupo: " + grupo.Key);
    foreach (var n in grupo)
        Console.WriteLine("  " + n);
}
// Saída esperada:
// Grupo: Ímpar
// 1
// 3
// 5
// Grupo: Par
// 2
// 4
// 6
```

Nesse código, `GroupBy(n => condição)` agrupa todos os elementos de acordo com a chave calculada (“Par” ou “Ímpar”). O resultado `grupos` é uma coleção de objetos que implementam `IGrouping<TKey, TElement>`. Cada objeto grupo possui uma propriedade `Key` (a chave do grupo) e pode ser iterado como uma coleção dos elementos correspondentes.

Internamente, o método `Enumerable.GroupBy` lê os elementos em um dicionário temporário de listas, agrupando aqueles que compartilham a mesma chave, e, em seguida, emite cada agrupamento como uma subcoleção com sua `Key`. Essa operação também segue o modelo de execução diferido (ver adiante), dessa forma o agrupamento só é efetivamente montado quando iteramos o resultado.

Vale destacar que `GroupBy` cria grupos do zero e não altera a sequência original; cada elemento original aparece exatamente em um dos grupos de saída.

Para consultas que precisam simplesmente verificar a existência de elementos, o LINQ fornece o operador quantificador `Any`. Ele retorna `true` se qualquer elemento na sequência satisfaz a determinada condição. Por exemplo:

```
int[] valores = { 2, 3, 4 };
bool existeTres = valores.Any(n => n == 3); // true
bool maiorQueDez = valores.Any(n => n > 10); // false
```

Aqui, `.Any(n => n == 3)` retorna verdadeiro porque existe um elemento igual a 3. `Any` retorna `true` se a expressão dada for verdadeira para pelo menos um elemento. De modo geral, `Any` com predicado é mais flexível que `Contains`, pois pode usar qualquer critério além de igualdade simples. Chamar `Any()` sem predicado verifica apenas se a sequência contém ao menos um elemento (ou seja, se não está vazia). Por exemplo, as chamadas a seguir retornariam `false` tanto se a sequência estivesse vazia quanto se não possuísse elementos que satisfizessem a condição:

```
bool algumPar = new int[] { }.Any(n => n % 2 == 0); // false (sequência vazia)
bool algum = new List<int>().Any(); // false (nenhum elemento)
```

O uso de `Any` é especialmente útil em subconsultas ou para checar a existência de itens antes de realizar outras operações, evitando lançar exceções ao acessar coleções vazias.

Um ponto fundamental em LINQ é o **diferimento de execução** das consultas. Isso significa que os operadores padrão de consulta não executam a operação no momento em que são chamados, mas somente quando a sequência é efetivamente enumerada. Veja a seguir:

```
var lista = new List<int>();
lista.Add(1);
// Monta a consulta, mas ainda não a executa
var query = lista.Select(n => n * 10);
lista.Add(2); // Altera a fonte APÓS montar a consulta

// A execução ocorre aqui, no foreach
foreach (int n in query)
    Console.WriteLine(n + "|");
// Saída: 10|20|
```

Nesse exemplo, embora a consulta `lista.Select(...)` tenha sido construída quando a lista continha apenas o elemento 1, o elemento 2 adicionado posteriormente está presente na saída final. Isso ocorre porque a projeção `Select` não avalia os elementos imediatamente: ela adia a execução até iterarmos sobre query no **foreach**.

Todos os operadores de consulta padrão fornecem execução diferida (lazy evaluation). A consulta só é executada (ou seja, varre a coleção e aplica filtros) no momento em que um iterador chama `MoveNext` (por exemplo, em um laço **foreach**).

Vale notar uma exceção: alguns operadores convertem a sequência em um tipo concreto ou valor escalar e então forçam a execução imediata, como fazem os métodos `Count()`, `ToList()`, `ToArray()` entre outros. Quando usamos `Count()`, o LINQ lê todos os elementos para retornar o número total, assim, não retorna um enumerador que possa diferir avaliação. Por isso, consultas como `int total = numeros.Where(n => n < 2).Count();` são executadas imediatamente, retornando o resultado no instante da chamada.

Em geral, o diferimento é importante porque permite montar consultas em etapas, reutilizá-las e compô-las (especialmente em LINQ to SQL/EF) sem imediatamente recuperar dados do banco até que isso seja necessário.



Observação

Em resumo, o LINQ combina operadores como `Where`, `Select`, `GroupBy` e `Any` para fornecer consultas expressivas e de fácil leitura. Cada operador atua de forma funcional, retornando novas coleções filtradas, projetadas ou agrupadas, sem modificar os dados originais.

Por fim, o modelo de execução diferida do LINQ é um aspecto-chave: ele separa a construção da consulta da sua execução real, conferindo flexibilidade e desempenho às operações de consulta em memória e em fontes externas de dados.

6.2 IEnumerable<T> x IQueryable (noções para acesso a dados)

Em LINQ, qualquer sequência que implemente `IEnumerable<T>` pode ser consultada com a mesma sintaxe de consulta típica (por exemplo, usando `Where`, `Select` etc.), e da mesma forma qualquer fonte de dados que implemente `IQueryable<T>` também aceita essas operações de consulta. Em outras palavras, o LINQ foi concebido para fornecer uma sintaxe comum para consultar dados em uma coleção, um documento XML, um banco de dados ou qualquer tipo que suporte as interfaces `IEnumerable<T>` ou `IQueryable<T>`.

Com isso, a mesma expressão de filtro ou projeção pode ser aplicada tanto em coleções locais, como `List<T>`, quanto em fontes remotas, como tabelas de banco de dados, simplificando o acesso aos dados.

Tecnicamente, `IEnumerable<T>` e `IQueryable<T>` têm propósitos distintos. A interface genérica `IEnumerable<T>` representa uma sequência de elementos sobre a qual se pode iterar (por exemplo, com `foreach`). As consultas LINQ sobre objetos que implementam `IEnumerable<T>` (ou seja, LINQ to Objects) são executadas localmente, em memória, e os operadores padrão correspondentes são encontrados na classe estática `Enumerable`. Isso significa que as expressões lambda usadas em consultas sobre `IEnumerable<T>` são convertidas em delegates que são executados diretamente no código IL do programa. Por exemplo, ao aplicar

```
List<Produto> lista = ObterListaProdutos();  
var caros = lista.Where(p => p.Preco > 100.0m);
```

a consulta `Where` filtra a lista em memória usando o predicado fornecido, iterando sobre todos os elementos locais. Nesse cenário, a execução da consulta é adiada até que se comece a iterar o resultado (deferred execution), mas todo o trabalho ocorre no lado do cliente (o próprio programa).

Já `IQueryable<T>` **estende** `IEnumerable<T>` com funcionalidades adicionais para representar consultas descritas como árvores de expressão (expression trees). Quando se realiza uma consulta LINQ sobre uma fonte de dados remota (por exemplo, uma tabela de banco de dados por meio do Entity Framework), a fonte implementa `IQueryable<T>` e os operadores padrão estão na classe estática `Queryable`. Nessa abordagem, em vez de executar o predicado como código local, o LINQ constrói uma árvore de expressão que representa a consulta. Essa árvore é então interpretada pelo provedor de consulta (por exemplo, o provedor do Entity Framework) para gerar uma instrução SQL equivalente, que será executada no banco de dados.

Em resumo, consultas interpretadas operam sobre sequências que implementam `IQueryable<>`, e elas resolvem para os operadores de consulta na classe `Queryable`, os quais emitem árvores de expressão que são interpretadas em tempo de execução. Esse mecanismo permite que o filtro ou a projeção definidos no código C# sejam traduzidos em consultas SQL executadas no servidor de dados, trazendo somente os resultados necessários. Por exemplo:

```
var clientesDoBrasil = contexto.Clientes  
    .Where(c => c.Pais == "BR" && c.Ativo == true);
```

Nesse caso, `contexto.Clientes` é um `IQueryable<Cliente>` (geralmente um `DbSet<Cliente>` do Entity Framework). A expressão lambda no `Where` não filtra imediatamente em memória; em vez disso, o provedor LINQ to Entities constrói uma expressão que será convertida em algo como **SELECT * FROM Clientes WHERE Pais = 'BR' AND Ativo = 1** no banco de dados. Assim, a execução ocorre no banco, retornando apenas os clientes brasileiros ativos, o que pode ser muito mais eficiente.

É importante notar que, embora `IQueryable<T>` herde de `IEnumerable<T>`, usar indevidamente operadores de `Enumerable` em uma fonte `IQueryable` pode forçar a execução da consulta no cliente. Os operadores de consulta em `Enumerable` podem até funcionar com sequências

`IQueryable`, mas as consultas resultantes sempre executam localmente no cliente, por isso existe um segundo conjunto de operadores de consulta na classe `Queryable`.

Em outras palavras, converter `IQueryable<T>` para `IEnumerable<T>` (por exemplo, usando `AsEnumerable()`) ou usar os métodos de extensão de `Enumerable` em vez de `Queryable` faz com que todo o conjunto de dados seja carregado em memória antes de aplicar o filtro. Portanto, para manter o processamento eficiente e no lado do servidor, deve-se usar os operadores de `Queryable` quando lidar com dados remotos.

Em termos de desempenho e cenários de uso, `IQueryable<T>` costuma ser preferível quando se trabalha com **fontes de dados externas ou muito volumosas**, pois permite que o próprio banco de dados execute parte do trabalho de filtro e agrupamento.

Já `IEnumerable<T>` é adequado para **coleções já carregadas em memória** ou para cenários em que não há suporte a consultas traduzíveis (por exemplo, em alguns serviços web ou em estruturas sem provedor de consulta).

De fato, algumas situações exigem converter explicitamente um `IQueryable<T>` em `IEnumerable<T>` – por exemplo, para usar lógica de negócio em LINQ to Objects sobre um subconjunto já obtido – usando métodos como `.ToList()` ou `.AsEnumerable()`. Contudo, é preciso cautela: realizar muitas operações em `IEnumerable<T>` após carregar tudo em memória pode levar a consumo elevado de memória e pior desempenho. Para ilustrar, considere o seguinte exemplo de código em C#:

```
// Fonte de dados local: List<T> implementa IEnumerable<T>
List<Pedido> todosPedidos = Repositorio.ObterPedidos();
IEnumerable<Pedido> pedidosRecentes =
    todosPedidos.Where(p => p.Data > DateTime.Today.AddMonths(-1));

// Fonte de dados remota: DbSet<T> implementa IQueryable<T>
var pedidosEF = contexto.Pedidos; // IQueryable<Pedido>
IQueryable<Pedido> pedidosEFRrecentes =
    pedidosEF.Where(p => p.Data > DateTime.Today.AddMonths(-1));
```

No primeiro caso, a consulta `Where` em `todosPedidos` será executada completamente em memória, filtrando a lista já carregada (LINQ to Objects). No segundo, `pedidosEFRrecentes` é ainda um `IQueryable<Pedido>` não executado; a consulta apenas é traduzida para SQL quando, por exemplo, se itera sobre ela ou chama-se algo como `ToList()`. Graças a isso, o Entity Framework enviará a instrução filtrada ao banco em vez de recuperar todos os pedidos e então filtrar localmente. As duas abordagens suportam a mesma sintaxe, mas uma extrai vantagem do mecanismo de consulta remoto.



Resumo

Esta unidade começou discutindo por que coleções e genéricos são centrais em C#. Mostramos que, na prática, o desenvolvedor esbarra primeiro em estruturas para guardar listas de itens e mapas de identificadores, e que arrays fixos ou coleções antigas baseadas em `object` trazem vários problemas: casts espalhados, exceções em tempo de execução, boxing e unboxing desnecessários e dificuldade de evolução do código.

As coleções genéricas do namespace `System.Collections.Generic`, como `List<T>` e `Dictionary<TKey, TValue>`, surgem como resposta a essas dificuldades, oferecendo segurança de tipos em tempo de compilação, melhor desempenho e um modelo uniforme baseado em `IEnumerable<T>` e iteração via `foreach`, muitas vezes com sequências geradas sob demanda por meio de iteradores com `yield return`.

Em seguida, falamos dos genéricos em sentido mais amplo, com foco em restrições e igualdade/ordenação. Definimos genéricos como mecanismo para escrever código que funciona com vários tipos sem perder segurança de tipos, e apresentamos o uso de `where` para restringir parâmetros de tipo: `class`, `struct`, `new()`, herança de uma classe base e implementação de interfaces específicas.

Também apresentamos o LINQ como mecanismo de consulta integrado à linguagem, apoiado em query expressions (`from`, `where`, `select`, `orderby` etc.) e nos standard query operators definidos em `System.Linq.Enumerable` e `System.Linq.Queryable`. Mostramos que consultas escritas na sintaxe de consulta são traduzidas pelo compilador em chamadas encadeadas a métodos como `Where`, `Select`, `OrderBy` e `GroupBy`, usando expressões lambda para fornecer a lógica de filtragem, projeção e agregação.

As lambdas são tratadas como funções anônimas convertidas em delegates (`Func` e `Action`), com exemplos simples de atribuição a `Func<int, int>` e de uso em consultas LINQ to Objects. O texto também discute closures, isto é, a captura de variáveis externas por lambdas, e explica a diferença entre expression lambdas (corpo de expressão única) e statement lambdas (corpo em bloco), sempre ressaltando o papel dos delegates e do tipo-alvo da lambda.

Por fim, expomos o risco de forçar execução no cliente ao usar operadores de `Enumerable` sobre fontes `IQueryable` e concluímos indicando cenários em que cada abordagem é mais adequada, enfatizando a importância de entender o modelo de execução para lidar bem com desempenho e acesso a dados.



Exercícios

Questão 1. Em um aplicativo de console em C#, um time precisa manter um catálogo de produtos que cresce ao longo da execução (sem saber previamente a quantidade), exibir os itens com `foreach` e permitir que o usuário pesquise rapidamente um produto pelo seu `Id`. A primeira versão usou um array fixo para o catálogo e, na busca por `Id`, acessou o dicionário pelo indexador sem tratar o caso de chave inexistente, gerando exceções. Considerando `List<T>`, `Dictionary<TKey, TValue>`, `IEnumerable<T>` e a busca segura por chave, qual conjunto de decisões é o mais coerente?

- A) Usar um `Produto[]` com tamanho grande por segurança, iterar com `for` e acessar `produtosPorId[id]` diretamente, pois exceções são o mecanismo padrão de controle de fluxo em buscas.
- B) Usar um `ArrayList` para permitir qualquer tipo de item, iterar com `foreach` e fazer cast para `Produto` na leitura, pois isso evita declarações genéricas e mantém o código mais simples.
- C) Usar um `Dictionary<int, Produto>` como única estrutura, porque ele substitui a `List<T>` em todos os casos e preserva, necessariamente, a mesma ordem de inserção na iteração.
- D) Usar uma `List<Produto>` para o catálogo (crescimento dinâmico), construir um `Dictionary<int, Produto>` para o acesso rápido por `Id`, exibir por um parâmetro `IEnumerable<Produto>` com `foreach` e buscar com `TryGetValue` para tratar chaves ausentes sem exceção.
- E) Usar uma `List<Produto>` para o catálogo, buscar pelo `Id` e percorrer a lista inteira com `foreach` a cada pesquisa, pois isso elimina a necessidade de um `Dictionary<TKey, TValue>`.

Resposta correta: alternativa D.

Análise das alternativas

A) Alternativa incorreta.

Justificativa: arrays têm tamanho fixo, o que contraria a necessidade de crescimento dinâmico. Além disso, acessar o dicionário por indexador sem tratamento de ausência de chave pode disparar uma exceção quando a chave não existe, enquanto `TryGetValue` evita esse problema.

B) Alternativa incorreta.

Justificativa: em coleções não genéricas como `ArrayList`, a leitura normalmente exige cast e pode falhar com `InvalidCastException` se o conteúdo não tiver o tipo esperado. Em tipos por valor, ainda há risco de boxing/unboxing, o que adiciona custo de alocação e cópia.

C) Alternativa incorreta.

Justificativa: um dicionário resolve bem o acesso por chave, mas não é uma substituição universal para a lista em todos os cenários. Além disso, a afirmação de que a iteração necessariamente preserva ordem de inserção não é uma garantia apresentada no conteúdo.

D) Alternativa correta.

Justificativa: `List<Produto>` resolve o crescimento sem tamanho prefixado. `Dictionary<int, Produto>` fornece busca rápida por chave. `IEnumerable<Produto>` permite que o método de exibição aceite diferentes coleções enumeráveis. `TryGetValue` trata a chave inexistente sem gerar exceção.

E) Alternativa incorreta.

Justificativa: a busca por varredura na lista funciona, mas ignora o objetivo de acesso rápido por Id. Para consultas repetidas, a combinação de lista (catálogo) com dicionário (índice por chave) atende melhor ao uso descrito.

Questão 2. Em um projeto de console em C#, um desenvolvedor escreveu uma consulta LINQ sobre uma `List<int>` para obter os quadrados dos números pares, mas decidiu adicionar novos valores à lista depois de montar a consulta. Ao executar o `foreach`, os novos valores também apareceram no resultado, contrariando a expectativa inicial. Considerando o diferimento de execução no LINQ e os operadores de execução imediata, qual ação é a mais coerente para obter um instantâneo dos resultados no momento em que a consulta foi definida?

A) Manter a consulta como `IEnumerable<int>` e confiar que `Where` e `Select` sempre executam no momento em que são chamados, pois as lambdas são avaliadas imediatamente.

B) Trocar o `Select` por `Aggregate`, pois os operadores de agregação retornam sequências e, por isso, preservam o diferimento até o `foreach`.

C) Substituir o `foreach` por um `for`, pois o `for` executa a consulta de forma imediata e o `foreach` é que causa o diferimento.

D) Armazenar a consulta em um `var` em vez de `IEnumerable<int>`, pois o uso de `var` elimina o diferimento e executa a consulta ao declarar a variável.

E) Converter o resultado da consulta para uma coleção materializada, como `ToList()` ou `ToArray()`, logo após montar o pipeline, para forçar a execução naquele instante.

Resposta correta: alternativa E.

Análise das alternativas

A) Alternativa incorreta.

Justificativa: em LINQ to Objects, operadores como `Where` e `Select` costumam ter execução adiada, de forma que a sequência é construída, mas só é avaliada quando é enumerada (por exemplo, em um `foreach`). Assim, alterações na fonte após a montagem podem refletir na enumeração.

B) Alternativa incorreta.

Justificativa: `Aggregate` é um operador de agregação que produz um valor (escalar), e não uma sequência. Além disso, agregações e conversões como `Count()`, `ToList()` e `ToArray()` são exemplos típicos de execução imediata, mas a alternativa descreve `Aggregate` como se retornasse sequência.

C) Alternativa incorreta.

Justificativa: o tipo de laço (`for` ou `foreach`) não elimina, por si só, o diferimento. O que dispara a execução é a enumeração da sequência. Ambos os laços podem enumerar e, portanto, ambos podem acionar a consulta.

D) Alternativa incorreta.

Justificativa: a indicação `var` é apenas inferência de tipo em tempo de compilação. O `var` não muda o modelo de execução do LINQ nem força materialização.

E) Alternativa correta.

Justificativa: operadores como `ToList()` e `ToArray()` materializam a sequência e forçam a execução imediatamente, produzindo uma coleção concreta com os elementos calculados naquele momento, sem depender de alterações posteriores na fonte.
